

Phase -1.3 — Non-Functional Requirements (NFR)

Objective

Functional Requirements define **what** the system should do.

Non-Functional Requirements (NFRs) define **how well** the system should perform while doing it.

These requirements influence almost every architectural decision made throughout the project. Production systems often fail not because they lack features, but because they cannot meet their non-functional requirements under real-world conditions.

Performance

NFR-1.1 API Response Time

Requirement

Under normal operating conditions:

- Simple read APIs should respond within **200 ms**.
- Complex operations should complete within **500 ms**.
- Booking requests may take longer due to transaction processing but should generally complete within **1 second**.

Why This Matters

Users perceive systems as fast when responses are received within a few hundred milliseconds. Longer delays increase abandonment rates and reduce overall user satisfaction.

Project Target

Since this is a learning project running on local hardware, the focus is on maintaining consistent performance rather than achieving enterprise-scale latency.

NFR-1.2 Concurrent Requests

Requirement

The system should correctly handle multiple users accessing the application simultaneously.

Project Target

The application should support:

- Multiple users browsing events.
- Multiple users booking different seats.
- Multiple users attempting to reserve the same seat.

Correctness is more important than raw throughput.

Scalability

NFR-2.1 Horizontal Scalability

Requirement

The backend should be designed so that multiple application instances can run simultaneously without modifying business logic.

Design Principles

- Stateless API servers.
- Shared PostgreSQL database.
- Shared Redis instance.
- JWT-based authentication.
- No in-memory session storage.

Why This Matters

Horizontal scaling is the most common strategy used by modern cloud platforms.

NFR-2.2 Vertical Scalability

Requirement

The application should also benefit from additional CPU and memory resources without requiring code changes.

Availability

NFR-3.1 Service Availability

Requirement

The application should remain operational as long as:

- PostgreSQL is available.
- Redis is available.
- The application server is running.

Future production deployments could introduce redundancy for each component.

NFR-3.2 Graceful Failure

Requirement

Unexpected failures should return meaningful error responses rather than exposing stack traces or crashing the application.

Examples include:

- Database unavailable.
 - Redis unavailable.
 - Invalid requests.
 - Authentication failures.
 - Internal server errors.
-

Reliability

NFR-4.1 No Double Booking

Requirement

The system must guarantee that only one successful booking can exist for a seat.

This is the most critical reliability requirement of the entire project.

NFR-4.2 Data Consistency

Requirement

Bookings, payments, and seat states must remain consistent even if failures occur during processing.

Transactions will be used to guarantee consistency.

NFR-4.3 Automatic Recovery

Requirement

Temporary seat holds should automatically expire without manual intervention.

Background workers will enforce this behavior.

Security

NFR-5.1 Authentication

Requirement

All protected endpoints must require a valid access token.

Unauthenticated users must receive appropriate authorization errors.

NFR-5.2 Authorization

Requirement

Administrative operations must only be accessible to users with administrative privileges.

NFR-5.3 Password Security

Requirement

Passwords must never be stored in plaintext.

Passwords must always be securely hashed before storage.

NFR-5.4 Token Security

Requirement

Refresh tokens should never be stored in plaintext.

Compromised database records should not expose active authentication credentials.

NFR-5.5 Environment Security

Requirement

Sensitive configuration must never be hardcoded.

Secrets should be loaded through environment variables.

Examples include:

- Database credentials.
 - JWT secrets.
 - Redis credentials.
 - API keys.
-

Maintainability

NFR-6.1 Modular Architecture

Requirement

Each feature should exist as an independent module with clear boundaries.

Changes to one module should have minimal impact on others.

NFR-6.2 Separation of Concerns

Requirement

Business logic, database access, validation, routing, configuration, and utilities should remain isolated from one another.

NFR-6.3 Readable Code

Requirement

Code should prioritize readability over cleverness.

Future developers should understand the implementation with minimal effort.

Observability

NFR-7.1 Structured Logging

Requirement

Every request should generate structured logs.

Logs should include:

- Timestamp
 - Request ID
 - User ID (when available)
 - HTTP method
 - Endpoint
 - Status code
 - Processing time
-

NFR-7.2 Metrics

Requirement

The system should expose operational metrics.

Examples include:

- Request count
 - Request duration
 - Error rate
 - Booking throughput
-

NFR-7.3 Health Monitoring

Requirement

Infrastructure should be able to determine whether the application is healthy.

This will later support:

- Docker health checks
 - Load balancers
 - Kubernetes probes
-

Testability

NFR-8.1 Unit Testing

Requirement

Business logic should be testable independently from external systems.

NFR-8.2 Integration Testing

Requirement

Critical API workflows should be verified using automated integration tests.

NFR-8.3 Repeatable Environment

Requirement

Every developer should be able to run the application using the same infrastructure.

Docker Compose will provide identical local environments.

Deployability

NFR-9.1 Containerization

Requirement

The application should be deployable using containers without modifying source code.

NFR-9.2 Continuous Integration

Requirement

Every commit should automatically validate:

- Build
- Linting
- Type safety
- Tests

Broken code should never reach the main branch unnoticed.

Extensibility

NFR-10.1 Future Integrations

Requirement

The architecture should support future integration with:

- Payment gateways
- Email providers
- SMS providers
- Notification services
- Analytics platforms

These additions should require minimal modification to existing business logic.

NFR-10.2 Feature Expansion

Requirement

The system should support future capabilities such as:

- QR ticket generation
- Multiple venues
- Multiple organizers
- Dynamic pricing
- Coupons
- Waiting lists

The initial architecture should not prevent these enhancements.

Operational Constraints

NFR-11.1 Development Environment

Requirement

The entire application should run locally using Docker Compose.

No paid cloud services are required.

NFR-11.2 Cost

Requirement

The complete project should remain free to build and operate during development.

NFR-11.3 Learning Priority

Requirement

Architectural correctness is prioritized over implementation speed.

Production engineering concepts should always take precedence over shortcuts.

Quality Attributes Summary

Quality Attribute	Priority
Reliability	Critical
Security	Critical
Maintainability	Critical
Scalability	High
Performance	High
Observability	High
Testability	High
Deployability	High
Extensibility	Medium
Cost Efficiency	Medium

Key Takeaways

The Functional Requirements describe **what** the application must accomplish.

The Non-Functional Requirements define the engineering standards the application must satisfy while accomplishing those goals.

Together, the FRs and NFRs form the foundation for every architectural decision that follows.

Deliverables

At the completion of this phase, the project has defined:

- Business functionality.
- Engineering quality standards.
- Operational expectations.
- Performance objectives.
- Security requirements.
- Scalability goals.
- Reliability guarantees.

The next phase, **Phase -1.4 — Actors, Roles & User Stories**, translates these requirements into real-world user interactions that will drive the API design and database model.